

The confusion comes down to a slight typo in the terminology, which completely changes the context: you are likely dealing with an **HTP Processor** (Hexagon Tensor Processor), not an "HTTP web processor". [1, 2]

If you are running Qualcomm's **QAIRT** (Qualcomm AI Runtime), **HTP** is the physical hardware chip (the NPU) inside the Snapdragon processor. Qualcomm explicitly names the software compiled to run on that chip the "**QNN HTP Backend**". [1, 3, 4, 5, 6]

The definitions clarify why this chip layer is referred to as a backend:

1. In AI/Hardware Engineering: The "HTP Backend"

In hardware acceleration (like Qualcomm QAIRT, Android NNAPI, or ONNX), the term **Backend** has a completely separate meaning from web development. [3, 5, 7, 8]

-
- **The Frontend:** The high-level software representation of your model (like a .dlc, GGUF, or ONNX file structure).
- **The Hardware Backend:** The low-level driver compiled to talk to a specific piece of silicon. Qualcomm provides a libQnnHtp.so file. This code acts as the "backend" to your inference engine because it compiles the mathematical graph down into native machine code that the Hexagon Tensor Processor (HTP) can physically execute. [1, 5, 9, 10]
-

2. In App Architecture: The "Server Backend"

In traditional app architecture, **Backend** means "the server side" that processes logic away from the screen. [11]

-
- Your **UI** (Jetpack Compose) sits on the user's screen.
- Your **Harness** handles the orchestration logic.
- If you point your harness at a local socket (like a port opened by llama.cpp or a local execution service), your app is treating that local engine like a **local loopback backend**. [11, 12]
-

How to Map This Out Without Naming Conflicts

To prevent confusion in your code when dealing with both local hardware chips and web services, categorize them based on their exact functions:

-
- **The UI (Front-End):** Your visual screens (Jetpack Compose). It has no business touching hardware math or web packets directly.
- **The Orchestrator (Harness):** Your background service that coordinates loops, handles memory, and routes tokens.

- **The Remote Backend:** Your cloud endpoints (like CloudFront or AWS Bedrock) reached via HTTP network packets.
- **The Local Inference Engines (Runtimes):** Your local execution binaries (llama.cpp or the QAIRT SDK).
- **The Hardware Backends:** The actual silicon drivers (Qualcomm **HTP** for the NPU, **GPU** drivers, or **CPU** loops) that execute the quantized model matrix math. [1, 3, 10, 12, 13]
-

Are you writing the configuration code right now to let users toggle between a local **QAIRT HTP execution** and a **Remote Cloud fallback**? If so, we can structure a clean configuration file that separates these exact definitions.

- [1] <https://docs.qualcomm.com>
- [2] <https://scalable.uni-jena.de>
- [3] <https://docs.qualcomm.com>
- [4] <https://docs.qualcomm.com>
- [5] <https://docs.qualcomm.com>
- [6] <https://www.cio.com>
- [7] <https://medium.com>
- [8] <https://pynomial.com>
- [9] <https://docs.qualcomm.com>
- [10] <https://docs.qualcomm.com>
- [11] <https://www.reddit.com>
- [12] <https://www.sciencedirect.com>
- [13] <https://martinfowler.com>